Day 4 Evidence Workbooklet

Comparisons and Boundary Cases

Engineering practice → threshold rule → Python comparison → boundary evidence → support route

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	______/40		Mastery check	secure / developing / needs support	

Yesterday the pump-flow intake values were converted from text into numeric fields. Today the team uses those typed values to check a quality boundary: a flow reading is acceptable if it is inside a target band. The exact boundary rows matter because a reading exactly at the lower or upper limit may be allowed depending on the operator.

The problem today is comparison evidence: below, at, or above a limit; inclusive versus exclusive rules; and an exact sentence that explains a boundary case.

Engineering task	Check whether converted pump-flow readings meet an acceptable range before the team uses them in a go/no-go gate.
Computational move	Evaluate one comparison at a time and explain exact-boundary cases without jumping ahead to compound decisions.
Python focus	`<`, `<=`, `>`, `>=`, `==`, `!=`, comparison result, inclusive limit, exclusive limit.
Evidence to keep	reading value, boundary value, operator, true/false result, and a sentence explaining whether the boundary is included.

Day 4 boundary rules

1. A comparison expression produces `True` or `False`.
2. `<=` and `>=` include equality at the boundary.
3. `<` and `>` do not include equality at the boundary.
4. `==` checks equality; it does not assign a value.
5. A reliable threshold note names the value, the limit, the operator, and the result.

1 Read the threshold notebook one comparison at a time

recognize predict

Use the converted numeric fields from the intake record. Do not combine the comparisons yet; Day 5 will do that.

```
# Converted numeric fields from the intake record
target_flow_lpm = 20.0
low_limit_lpm = 19.6
high_limit_lpm = 20.4
reading_low_edge = 19.6
reading_middle = 20.0
reading_high_edge = 20.4
reading_high_miss = 20.5

# Individual comparison checks
reading_low_edge >= low_limit_lpm
reading_low_edge > low_limit_lpm
reading_high_edge <= high_limit_lpm
reading_high_edge < high_limit_lpm
reading_high_miss <= high_limit_lpm
reading_middle == target_flow_lpm
```

Pts.	Prompt	My evidence
1	Which variables define the lower and upper allowed flow limits?	
1	Which reading is exactly equal to the lower limit?	
1	Which reading is exactly equal to the upper limit?	
1	Which reading is above the upper limit?	

2 Predict comparison results at exact boundaries

predict explain

Complete the table. Explain the result using the operator, not just the number.

Pts.	Comparison expression	True or False?	Boundary reason
2	`reading_low_edge >= low_limit_lpm`		
2	`reading_low_edge > low_limit_lpm`		
2	`reading_high_edge <= high_limit_lpm`		
2	`reading_high_edge < high_limit_lpm`		

Common trap to check

A reading exactly at the limit is included by `<=` or `>=`, but it is not included by `<` or `>`. The operator decides the boundary, not the word "limit" by itself.

3 Separate equality from assignment and mismatch checks

recognize predict

Python uses == for equality checks and = for assignment. This page keeps comparison evidence separate from stored state.

Pts.	Expression or line	Result or effect	What evidence does it create?
2	`reading_middle == target_flow_lpm`		
2	`reading_high_miss != target_flow_lpm`		
2	`reading_middle = target_flow_lpm`		
2	`reading_high_miss <= high_limit_lpm`		

Pts.	Prompt	My response
2	Why is == a safer symbol than the word "is" when writing comparison evidence?	
2	Why should a threshold note not say only "near the target"?	

4 Build a boundary table before combining conditions

trace explain

Complete the boundary table. The last column should be a short engineering sentence, not only `True` or `False`.

Pts.	Reading	Lower check: `>= 19.6`	Upper check: `<= 20.4`	Boundary evidence sentence
2	`19.5`			
2	`19.6`			
2	`20.4`			
2	`20.5`			

Bridge to Day 5

Tomorrow the team will combine separate comparison results into a go/no-go expression. Today the important habit is to make each comparison traceable first: value, operator, limit, result, and boundary sentence.

5 Debug a boundary claim

debug test explain

A teammate writes this note after checking the lower edge reading:

Boundary note to debug

The `19.6` L/min reading fails because it is not greater than the lower limit of `19.6` L/min.

Pts.	Prompt	My response
2	What part of the teammate's sentence uses a real comparison result?	
2	What rule did the teammate use that may not match the allowed boundary?	
2	Write the inclusive comparison that would allow the exact lower-limit reading.	
2	Rewrite the boundary note so it names the operator and explains whether the exact limit is included.	

Boundary-test habit

When a threshold rule matters, test at least three rows: just below the boundary, exactly at the boundary, and just above the boundary. Exact-boundary rows catch many comparison mistakes.

6 Mastery check and support route

transfer explain

Use your own evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example where changing from `>` to `>=` changes the result at an exact boundary.	
2	In one or two sentences, explain why Day 3 type/cast evidence is needed before Day 4 boundary checks.	

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can evaluate comparison operators and explain exact lower or upper boundary cases.
☐	Developing: I can evaluate some comparisons, but I still need support with inclusive versus exclusive limits.
☐	Needs support: I need a smaller example before I can trust my boundary-check evidence.

My next support move

My issue is mostly: setup / Python syntax / tracing / operator choice / boundary cases / confidence / time.
The first boundary where I got uncertain was: ______________________________
My next small action is: ask a partner / ask instructor / rebuild the boundary table / test just-below-at-above rows / try the recovery version.

Workbooklet target: I can evaluate Python comparison operators, explain exact-boundary cases, and prepare separate comparison evidence before Boolean combinations.